Submission to TCS 2004 Track 2: Logic, Semantics, Specification and Verification

title. **Implementing a bisimulation-checker in FreshOCaml**

author. Murdoch J. Gabbay
affil. LIX
email. gabbay@lix.polytechnique.fr
tel. +33 1 69 33 34 79
fax. +33 1 69 33 30 14
addr. Laboratoire d'Informatique (LIX), École Polytechnique,
91128 Palaiseau Cedex, France.

author. Lucian Wischik
affil. Microsoft Corporation
email. lu@wischik.com

Abstract. We have implemented several different formalisms of the pi calculus and its late bisimulation. Unexpectedly, even very different formalisms turned out to have almost identical code—notably the original due to Milner, one based on FM-theory due to Gabbay, and one based on Higher-Order Abstract Syntax due to Miller and Tiu.

Implementing a bisimulation-checker in FreshML

Murdoch J. Gabbay (INRIA)
Laboratoire d'Informatique (LIX), École Polytechnique
91128 Palaiseau Cedex - France
gabbay@lix.polytechnique.fr

Lucian Wischik
Microsoft Corporation
lu@wischik.com

1 Introduction

In codice, veritas. There have been numerous different 'mathematical' presentations of the pi-calculus and its theory of bisimulation (e.g. [2, 5, 6, 9]). Our intuition was that one particular presentation, that of Gabbay [2] was better than the others—cleaner and simpler. To demonstrate this we turned several presentations into actual implementations. Our intuition turned out false: despite its unconventional mathematical foundation Gabbay's yielded code which was little different from the others.

Our implementations effectively constitute half of a model-checker: the code for terms in the pi-calculus (i.e. states), and for labelled transitions between terms, and for checking whether the transitions out of one term match those out of another. These are precisely the areas that one finds in a mathematical presentation of the pi-calculus. The other half of the model checker, which we have not implemented, involves generating a graph of transitions and collapsing the duplicate states. We have been purposefully naïve in our code—it stays close to the spirit and the letter of the original mathematical formalisms.

It is worth noting that the bisimulation checker in this paper are very simple. Each fits on a single page of A4, albeit in a small font size. Implementing a bisimulation-checker for pi might be considered as a standard undergraduate exercise, alongside implementing a combinator machine for the lambda-calculus.

Methodologically, one must ask whether implementing a formalism is a valid approach, and whether it has significance. For validity, our idea was that a formalism's mathematical clarity should be reflected in the clarity of its code; its formality shoudl be reflected in how close its code is to its maths; its abstractness should be reflected in how modular the code is. For significance, we found that the implementations gave us a deeper understanding of pi-calculus semantics in a way which theory alone did not. This paper attempts to share that understanding. In particular we wish to communicate our excitement at the unexpected result—that the diffrerences in theory turned out at the programming level. In effect, the implementations are a *denotation* for the formalisms!

We remark on our implementation of Miller and Tiu's presentation of the pi-calculus [5]. They define pi-calculus transitions and bisimulation in a particular form of logic, Higher-Order Abstract Syntax (HOAS). Because of this, bisimulation statements can be verified with a theorem-prover for that logic. We came to understand the logic's primitives as an *Application Programming Interface* (API) layer between the pi-calculus and a theorem-prover. When we implemented the primitives using tuples, the resulting program was more-or-less the same as our other implementation. When

we implemented them using features provided by our functional language (FreshOCaml [1]) the resulting program was somewhat different.

Background

The pi-calculus was introduced by Milner, Parrow, and Walker in [6], in 1992. Its operational semantics are described through a *labelled transition system*—the labelled transition $P \xrightarrow{\alpha} P'$ means 'a term P can perform the action α, so evolving to P'. The action α may be the emission of data, the reception of data, or an internal move (called τ).

The pi-calculus uses a notion of behavioural equivalence called a *bisimulation relation*—for P and Q to be bisimilar then any $P \xrightarrow{\alpha} P'$ must be matched by a $Q \xrightarrow{\alpha} Q'$ such that P' and Q' are themselves bisimilar.

These simple-looking definitions are subtle notably because of fresh names: when P emits a freshly generated name then Q should be able to emit *any other* freshly generated name rather than specifically the same one; however when P receives data then Q should be able to receive that same data. The various alternative presentations of the pi-calculus stem largely from attempts to make these matters more clear, more formal, or more abstract.

Plan of paper.

Section 2 gives the original pi-calculus semantics, and a direct implementation of in in the OCaml language. We remark that some effort is needed o account for bound names, alpha-renaming and capture-avoiding substitution. To avoid this effort in the rest of the paper we then switch to Fresh OCaml, whose built-in features provide a more elegant treatment of bound names. Section 3 considers what happens if we drop bound output transitions and use FM binding implement them instead, as discussed in [2]. The subsequent sections cover alternative semantics of Sewell and Miller/Tiu.

2 Original pi calculus

In (1) we briefly give the traditional presentation of pi-calculus transitions, following [6, Table 2, p.9]. It is a relation $\rightarrow \subseteq \Pi \times Act' \times \Pi$ where Π and Act' are datatypes of process and actions and actions respectively inductively defined by

$$\begin{array}{c} \Pi \stackrel{\text{def}}{=} 1 + \mathbb{A}^2 \times \Pi + \mathbb{A}^2 \times \Pi + \Pi^2 + \mathbb{A} \times \Pi \\ P ::= 0 \mid \overline{a}b.P \mid a(b).P \mid (P \mid P) \mid \nu(n)P \\ Act' = 1 + \mathbb{A}^2 + \mathbb{A}^2 + \mathbb{A}^2 \qquad \alpha ::= \tau \mid \overline{a}b \mid \overline{a}(b) \mid ab \end{array} \tag{1}$$

The reductions $\rightarrow$ are inductively defined as follows:

$$\overline{a}b.P \xrightarrow{\overline{a}b} P \qquad a(c).P \xrightarrow{ac} P \;\; c \neq a \qquad \frac{P \xrightarrow{\alpha} P'}{\nu(d).P \xrightarrow{\alpha} \nu(d).P'}\, d \notin n(\alpha)$$

$$\frac{P \xrightarrow{\overline{a}b} P' \quad Q \xrightarrow{ac} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'\{b/a\}} \qquad \frac{P \xrightarrow{\overline{a}b} P'}{\nu(b).P \xrightarrow{\overline{a}(b)} P'}\, a \neq b \qquad \tau.P \xrightarrow{\tau} P \tag{2}$$

$$\frac{P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q}\, bn(\alpha) \cap fn(Q) = \emptyset \qquad \frac{P \xrightarrow{\overline{a}(b)} P' \quad Q \xrightarrow{ab} Q'}{P \mid Q \xrightarrow{\tau} \nu(b).(P' \mid Q')} \qquad \frac{P \mid !P \xrightarrow{\alpha} Q}{!P \xrightarrow{\alpha} Q}$$

```
(* pi-calculus in original form.
   We use ML state (unit ref) to generate fresh names
   *)

type atom=unit ref;;

type proc = Nil
| Snd of atom*atom*proc | Rcv of atom*atom*proc
| Par of proc*proc | New of atom*proc | Rep of proc;;

let rec subs a b p = match p with
  Snd(x,y,p) -> Snd(subsA a b x,subsA a b y,subs a b p)
| Rcv(x,n,p) -> if a=n then Rcv(x,n,p) else
                 if b=n then let n'=ref () in
Rcv(subsA a b x,n',subs a b (subs n n' p))
                 else
        Rcv(subsA a b x,n,subs a b p)
| Par(p1,p2) -> Par(subs a b p1,subs a b p2)
| New(n,p)   -> if a=n then New(n,p)
                       else
                if b=n then let n'=ref () in
                 New(n',subs a b (subs n n' p))
                       else New(n,subs a b p)
| Rep(p)     -> Rep(subs a b p)
| Nil        -> Nil
and subsA a b x = if x=a then b else x;;

let rec list_remove a = function [] -> []
| x::l' -> if a=x then (list_remove a l')
                  else a::(list_remove a l');;

let rec names_in = function
  Nil           -> []
| Snd(a,b,p)    -> a::b::(names_in p)
| Rcv(a,n,p)    -> a::(list_remove n (names_in p))
| Rep(p)        -> names_in p;
| New(n,p)      -> list_remove n (names_in p)
| Par(p,q)      -> (names_in p) @ (names_in q);;

type evol =
  Etau of proc | Esnd of atom*atom*proc
| Ebsnd of atom*atom*proc | Ercv of atom*atom*proc;;

let par_rule q = function
  Etau (p')      -> Etau(Par(p',q))
| Esnd (a,b,p')  -> Esnd(a,b,Par(p',q))
| Ebsnd (a,b,p') -> if List.mem b (names_in q)
                 then let b'=ref () in
                      Ebsnd(a,b',Par(subs b b' p',q))
                 else Ebsnd(a,b,Par(p',q))
| Ercv (a,b,p')  -> if List.mem b (names_in q)
                 then let b'=ref () in
     Ercv(a,b',Par(subs b b' p',q))
else Ercv(a,b,Par(p',q));;

let com_rule  = function
  (Esnd (a1,b1,p1'), Ercv (a2,b2,p2')) ->
      if a1=a2 then [Etau(Par(p1',subs b2 b1 p2'))]
               else []
| (Ercv (a1,b1,p1'), Esnd (a2,b2,p2')) ->
      if a1=a2 then [Etau(Par(subs b1 b2 p1',p2'))]
               else []
| (Ebsnd (a1,b1,p1'), Ercv (a2,b2,p2')) ->
   if a1=a2 then let n=ref () in
   [Etau(New(n,Par(subs b1 n p1',subs b2 n p2')))]
            else []
| (Ercv (a1,b1,p1'), Ebsnd (a2,b2,p2')) ->
   if a1=a2 then let n=ref () in
   [Etau(New(n,Par(subs b1 n p1',subs b2 n p2')))]
            else []
| _ -> [];;

let open_rule b = function
   (Esnd (a,b',p')) -> if b=b' && a!=b then
                         [Ebsnd(a,b,p')] else []
| _ -> [];;

let res_rule b = function
  Esnd (a,b',p')  -> if a!=b && b!=b' then
                       [Esnd(a,b',p')] else []
| Ebsnd (a,b',p') -> if a!=b then if b'=b
   then let n=ref () in [Ebsnd(a,n,subs b n p')]
   else [Ebsnd(a,b',p')]
                                else []
| Ercv (a,b',p')  -> if a!=b then if b'=b
   then let n=ref () in [Ercv(a,n,subs b n p')]
   else [Ercv(a,b',p')]
                                else []
| Etau (p')       -> [Etau(p')];;

let rec multiply_lists l1 l2 =
  match (l1, l2) with
   ([], l2)         -> []
| ([a1],a2::l2)     -> (a1,a2)::(multiply_lists [a1] l2)
| ([a1],[])         -> []
| (a1::a2::l1, l2) -> (multiply_lists [a1] l2) @
                       (multiply_lists (a2::l1) l2);;

let flatten_map f l = List.flatten (List.map f l);;

let rec evol_of = function
  Nil          -> [ ]
| Snd(a,b,p)   -> [Esnd(a,b,p)]
| Rcv(a,b,p')  -> [Ercv(a,b,p')]
| New(b,p)     ->
   (flatten_map (open_rule b) (evol_of p))@
   (flatten_map  (res_rule b) (evol_of p))
| Par(p,q)     ->
   let (evolp,evolq) = (evol_of p,evol_of q) in
        (List.map (par_rule q) evolp)
     @ (List.map (par_rule p) evolq)
     @ (flatten_map com_rule
          (multiply_lists evolp evolq))
| Rep(p)         ->
    let evolp = evol_of p
     in (List.map (par_rule (Rep(p))) evolp)
      @ (List.map (par_rule (Rep(p)))
          (flatten_map com_rule
    (multiply_lists evolp evolp))
);;

let rec is_bisim p q =
  (is_bisims4(evol_of p, evol_of q)) &&
  (is_bisims4(evol_of q, evol_of p))
and
is_bisims4 (l1,l2) = (List.for_all (function e1 ->
                      List.exists (function e2 ->
                      is_bisims2(e1,e2)) l2) l1)
and
is_bisims2  = function
  (Etau(p),Etau(q))             -> (is_bisim p q)
| (Esnd(a1,b1,p),Esnd(a2,b2,q)) ->
   if a1=a2 && b1=b2 then (is_bisim p q) else false
| (Ercv(a1,c1,p),Ercv(a2,c2,q)) ->
       (if a1=a2 then let c=ref() in
          (is_bisim (subs c1 c p) (subs c2 c q))
        else false)
    && List.for_all (function x -> x)
        (List.map
         (function b -> is_bisim (subs c1 b p)
                          (subs c2 b q))
          ((names_in p) @ (names_in q))
        )
| (Ebsnd(a1,c1,p),Ebsnd(a2,c2,q)) ->
      if a1=a2 then let c=ref () in
         (is_bisim (subs c1 c p) (subs c2 c q))
       else false
| _ -> false;;

(* A brief demonstration... *)
let (a,b,u) = (ref (), ref(), ref());;
let p = Rcv(a,u,Nil);;
let q = Snd(a,b,Nil);;
evol_of p;;
evol_of q;;
evol_of (Par(p,q));;
evol_of (New(a,Par(p,q)));;
is_bisim (Par(p,q)) (Par(q,p));;
```

Fig. 1. A program for the pi-calculus following the original 1992 and written in plain OCaml

We omit symmetric versions.

We implement this mathematics in OCaml in Figure 1. The code is mostly straightforward: in the code, `evol_of` generates all evolutions $\xrightarrow{\alpha} P'$ from a term P; it invokes subfunctions `par_rule`, `com_rule`, `open_rule` ... which correspond directly to the rules of (2); and `is_bisim` checks that two terms P and Q have the same evolutions. We remark only on `rep_rule`. As written in Figure 1 it would lead immediately to infinite recursion. We have instead used the standard derived replication rules: a transition of $P|!P$ came either from P alone, $!P \xrightarrow{\alpha} P'|!P$, or from an interaction with itself $!P \xrightarrow{\tau} P'|P''|!P$.

The code is not purely functional; it uses `unit ref` to generate fresh names, for instance in capture-avoiding substitution. We could have made the code purely functional by carrying around an explicit state, and generating names fresh with respect to this state. But there seemed little point, since `unit ref` does that anyway.

FreshOCaml

We have also implemented the same pi calculus in FreshOCaml (Figure 2). FreshOCaml is a conservative extension of OCaml which adds *bindable types*. These simplify the implementation of calculi with bound names. The practical effect is to simplify the implementation of substitution (the function `subs`) and clash-avoidance (the function `names_in`). We give here a brief practical tutorial on FreshOCaml; for more information, see [1, 10].

The following declares that 'atom' is a bindable type:

```
type atom=unit name;;
```

(Bindable types are piggy-backed onto a base type, in this case 'unit'. It is done this way because piggy-backed types are easy to add to OCaml. The base type is not important, and indeed is not present in FreshML). A bindable type behaves much like `unit ref` in ML. The underlying implementation of `a:atom` is (say) an integer. Programs evolve in a global state containing an integer counter; each time a fresh name generated the value of the counter is used to implement it, and the counter is incremented. The following expression will inject a fresh name into the state:

```
let n=(fresh:atom) in ...
```

The significant difference from `unit ref` is that the declaration of `atom` also introduces a polymorphic **swapping function**:

```
swap : atom -> atom -> 'X -> 'X
```

The literal operational meaning of `swap a and b in t` in the compiler is to swap `a` and `b` in the underlying representation of `t`. (If `'X` is a complex data object, the swapping might be implemented lazily for efficiency.)

When a bindable-type 'atom' is declared, FreshOCaml also generates a type-constructor `<<atom>>'X` for **abstractions** of atom. The abstraction `<<a>>t` is implemented internally as a pair `(a,t)`. However, it can only be pattern-matched by **concreting** it, as for example in the 'unpacking' below:

```
let unpack = function <<a>>t -> (a,t);;
```

This works as follows. When matching an abstraction whose internal implementation is `(n,exp)`, against this pattern `<<a>>t`, the compiler generates a fresh atom `m` and binds `m` to `a` and `t` to `swap m n exp`. Thus `unpack <<a>>>t` evaluates to `(m,swap m n t)` for some fresh `m`.

Abstractions and concretions make it easier to work with fresh and bound names. This is particularly evident in the code for function `subs` in Figure 2, which we describe mathematically now. Substitution $x\{a/b\}$ on atoms is as normal: x if $x \neq b$ and a otherwise. Substitution on terms $P\{a/b\}$ is as below. Capture-avoidance comes for free: there is no need to explicitly generate a fresh name in the bound-input or restriction case, since a fresh name is implicitly generated by concreting. (Quantifiers, when omitted, are universal and outermost. The quantifier $\mathit{M}n.$ is pronounced 'for fresh n' and mathematically means *for all but finitely many* (this provides a 'context-free' notion of 'fresh for', since any context, if it is of syntax or syntax-like objects, will only mention finitely many names [3, 10]).

$$\begin{gathered} 0\{a/b\} = 0 \quad (P \mid Q)\{a/b\} = (P\{a/b\}) \mid (Q\{a/b\}) \\ (\overline{x}y.P)\{a/b\} = \overline{x\{a/b\}}y\{a/b\}.(P\{a/b\}) \\ \forall x.\ \mathit{M}n.\ \forall P.\ (x\langle n\rangle.P)\{a/b\} = x\{a/b\}\langle n\rangle.(P\{a/b\}) \\ (!P)\{a/b\} = !(P\{a/b\}) \quad \mathit{M}n.\ \forall P.\ (\nu\langle n\rangle.P)\{a/b\} = \nu\langle n\rangle.(P\{a/b\}). \end{gathered} \tag{3}$$

Finally, bisimulation is coinductively defined like this:

$$P \simeq Q \implies \forall \alpha, P'.\ P \xrightarrow{\alpha} P' \implies \begin{cases} \exists Q'.\ Q \xrightarrow{\alpha} Q' \wedge P' \simeq Q' & \text{if } \alpha = \overline{a}b \text{ or } \alpha = \tau \\ \exists Q', n.\ n \notin fn(P,Q) \cup \{a,b\} \wedge Q \xrightarrow{\overline{a}(n)} Q' \wedge P'\{n/b\} \simeq Q' & \text{if } \alpha = \overline{a}(b) \\ \exists Q'.\ \forall b.\ Q \xrightarrow{\alpha} Q' \wedge P'\{b/c\} \simeq Q'\{b/c\} & \text{if } \alpha = ac \end{cases} \tag{4}$$

We implement this mathematics in OCaml in Figure 1. Compare this with Figure 2. This is the same mathematics but implemented delegating α-equivalence to FreshOCaml.

The two programs correspond line-for-line. `ref ()` in the first corresponds to `fresh` in the second (we may specify `fresh:atom` but the interpreter would infer that anyway). We use colour highlighting. So we see that we have exploited some ML state to generate fresh names. The only significant difference is that in the first version we often have to explicitly freshen bound names; the clauses `Rcv` and `New` in `subs`, `Ebsnd` and `Ercv` in `par_rule`, `com_rule`, and `res_rule`, and the `(Ercv,Ercv)` clause of `is_bisims2`.

Unfortunately FreshOCaml does not allow non-linear patterns over names, which means that the pattern `(<<n>>t,<<n>>t')` for choosing the *same* bound name for a pair of abstractions, is not possible. This is why renamings still appear for example in the `Ebsnd` clause of `com_rule` and the `(Ercv,Ercv)` clause of `is_bisims2`.

Sometimes we use `swap` in the FreshOCaml version. For a name which we have just generated fresh, swapping and substitution are equivalent. However, swapping is probably more efficient in FreshOCaml because firstly it is primitive (and not defined, e.g. by induction on particular types) and second swappings are distributed into the calculation lazily while substitutions are evaluated call-by-value according to the standard ML evaluation strategy.

Some of this is not necessary in the FreshOCaml version becuase in pattern-matching `<<n>>t` the compiler chooses n fresh names. `par_rule`, for example, comes out significantly cleaner.

Note also that `list_remove` and `names_in` appear earlier in the OCaml version: they are simply needed earlier for renamings in `par_rule`. We could fix that by taking advantage of ML's global state and generating `unit ref` as needed.

A problem with FreshOCaml is that because there is no canonical ordering on names, it is hard to write efficient lookup functions on tables indexed by names. These and other problems are being

```
(* pi-calculus in original form.
   We use FreshOCaml atom-abstractions
   *)

type atom=unit name;;  (* Create a type of names *)

type proc = Nil        (* Use atoms-abstractions *)
| Snd of atom*atom*proc | Rcv of atom*(<<atom>>proc)
| Par of proc*proc | New of <<atom>>proc
| Rep of proc;;

let rec subs a b p = match p with
     Snd(x,y,p)    ->
       Snd(subsA a b x,subsA a b y,subs a b p)
| Rcv(x,<<n>>p) ->
       Rcv(subsA a b x,<<n>>(subs a b p))
| Par(p1,p2)    -> Par(subs a b p1,subs a b p2)
| New(<<n>>p)   -> New(<<n>>(subs a b p))
| Rep(p)        -> Rep(subs a b p)
| Nil           -> Nil
and  subsA a b x = if x=a then b else x;;

type evol =
  Etau of proc | Esnd of atom*atom*proc
| Ebsnd of atom*(<<atom>>proc)
| Ercv of atom*(<<atom>>proc);;

let par_rule q = function
  Etau (p')         -> Etau(Par(p',q))
| Esnd (a,b,p')     -> Esnd(a,b,Par(p',q))
| Ebsnd (a,<<b>>p') -> Ebsnd(a,<<b>>(Par(p',q)))
| Ercv (a,<<b>>p')  -> Ercv(a,<<b>>(Par(p',q)));;

let com_rule  = function
  (Esnd (a1,b1,p1'), Ercv (a2,<<b2>>p2')) ->
    if a1=a2 then [Etau(Par(p1',subs b2 b1 p2'))]
             else []
| (Ercv (a1,<<b1>>p1'), Esnd (a2,b2,p2')) ->
    if a1=a2 then [Etau(Par(subs b1 b2 p1',p2'))]
             else []
| (Ebsnd (a1,<<b1>>p1'), Ercv (a2,<<b2>>p2')) ->
    if a1=a2 then let n=(fresh:atom) in
[Etau(New(<<n>>(Par(subs b1 n p1',subs b2 n p2'))))]
             else []
| (Ercv (a1,<<b1>>p1'), Ebsnd (a2,<<b2>>p2')) ->
    if a1=a2 then let n=(fresh:atom) in
Etau(New(<<n>>(Par(subs b1 n p1',subs b2 n p2'))))]
             else []
| _ -> [];;

let open_rule b = function
  (Esnd (a,b',p')) -> if b=b' && a!=b then
                   [Ebsnd(a,<<b>>p')] else []
| _ -> [];;

let res_rule b = function
  Esnd (a,b',p')  -> if b!=b' && a!=b then
                      [Esnd(a,b',p')] else []
| Ebsnd (a,phat') -> if a!=b then
           [Ebsnd(a,phat')] else []
| Ercv (a,phat')  -> if a!=b then
             [Ercv(a,phat')] else []
| Etau (p')       -> [Etau(p')];;

let rec multiply_lists l1 l2 =
  match (l1, l2) with
  ([], l2)         -> []
| ([a1],a2::l2)    -> (a1,a2)::(multiply_lists [a1] l2)
| ([a1],[])        -> []
| (a1::a2::l1, l2) -> (multiply_lists [a1] l2) @
                      (multiply_lists (a2::l1) l2);;

let flatten_map f l = List.flatten (List.map f l);;

let rec evol_of = function
  Nil           -> [ ]
| Snd(a,b,p)    -> [Esnd(a,b,p)]
| Rcv(a,phat')  -> [Ercv(a,phat')]
| New(<<b>>p)   ->
   (flatten_map (open_rule b) (evol_of p))@
   (flatten_map  (res_rule b) (evol_of p))
| Par(p,q)      ->
   let (evolp,evolq) = (evol_of p,evol_of q) in
        (List.map (par_rule q) evolp)
     @ (List.map (par_rule p) evolq)
     @ (flatten_map com_rule
         (multiply_lists evolp evolq))
| Rep(p)        ->
    let evolp = evol_of p
     in (List.map (par_rule (Rep(p))) evolp)
      @ (List.map (par_rule (Rep(p)))
          (flatten_map com_rule
    (multiply_lists evolp evolp))
);;

let rec list_remove a = function [] -> []
| x::l' -> if a=x then (list_remove a l')
                  else a::(list_remove a l');;

let rec names_in = function
  Nil           -> []
| Snd(a,b,p)    -> a::b::(names_in p)
| Rcv(a,<<n>>p) -> a::(list_remove n (names_in p))
| Rep(p)        -> names_in p;
| New(<<n>>p)   -> list_remove n (names_in p)
| Par(p,q)      -> (names_in p) @ (names_in q);;

let rec is_bisim p q =
  (is_bisims4(evol_of p, evol_of q)) &&
  (is_bisims4(evol_of q, evol_of p))
and
is_bisims4 (l1,l2) = (List.for_all (function e1 ->
                       List.exists  (function e2 ->
      is_bisims2(e1,e2)) l2) l1)
and
is_bisims2  = function
  (Etau(p),Etau(q))             -> (is_bisim p q)
| (Esnd(a1,b1,p),Esnd(a2,b2,q)) ->
   if a1=a2 && b1=b2 then (is_bisim p q)
                     else false
| (Ercv(a1,<<c1>>p),Ercv(a2,<<c2>>q)) ->
       (if a1=a2 then let c=(fresh:atom) in
(is_bisim (swap c and c1 in p) (swap c and c2 in q))
         else false)
    && List.for_all (function x -> x)
       (List.map
        (function b -> is_bisim (subs c1 b p)
                        (subs c2 b q))
         ((names_in p) @ (names_in q))
       )
| (Ebsnd(a1,<<c1>>p),Ebsnd(a2,<<c2>>q)) ->
       if a1=a2 then let c=(fresh:atom) in
          (is_bisim (swap c and c1 in p)
                    (swap c and c2 in q))
                else false
| _ -> false;;

(* A brief demonstration... *)
let (a,b,u) = (fresh, fresh, fresh);;
let p = Rcv(a,u,Nil);;
let q = Snd(a,b,Nil);;
evol_of p;;
evol_of q;;
evol_of (Par(p,q));;
evol_of (New(<<a>>(Par(p,q))));;
is_bisim (Par(p,q)) (Par(q,p));;
```

Fig. 2. A program for the pi-calculus following the original 1992, written in FreshOCaml

worked on in Cambridge now; for our purposes we simply note that delegating α-equivalence to FreshOCaml makes our programs a little cleaner so we take advantage of this in future.

3 Pi calculus in FM

We recall the FM presentation of the pi-calculus, from [2]. Define the **terms** and **actions** as follows.

$$\begin{aligned} \Pi &\cong 1 + \mathbb{A}^2 \times \Pi + \mathbb{A} \times \langle\mathbb{A}\rangle\Pi + \Pi + \Pi^2 + \langle\mathbb{A}\rangle\Pi \\ P &::= 0 \mid \overline{a}b.P \mid a\langle b\rangle.P \mid !P \mid (P \mid P) \mid \nu\langle a\rangle P \\ Act &\cong 1 + \mathbb{A}^2 + \mathbb{A}^2 \qquad \alpha, \mu ::= \tau \mid \overline{a}b \mid ab \end{aligned} \tag{5}$$

We defined **name substitution** in (3).

The labelled transition relation is $\rightarrow \subseteq \Pi \times \langle\mathbb{A}\rangle(Act \times \Pi)$. We write

$$P \overset{c.\alpha}{\rightarrow} P' \quad \text{for} \quad \langle P, \langle c\rangle\langle \alpha, P'\rangle\rangle \in \rightarrow.$$

The abstracted FM name c models name-generation. For example, $\nu\langle c\rangle.\overline{a}c \overset{c.\overline{a}c}{\rightarrow} 0$ represents $\nu(c).\overline{a}c \overset{\overline{a}(c)}{\rightarrow} 0$ and $\nu(c).\overline{a}c \overset{\overline{a}(d)}{\rightarrow} 0$ and $\nu(c).\overline{a}c \overset{\overline{a}(e)}{\rightarrow} 0 \ldots$. Call $\langle c\rangle\langle \alpha, P'\rangle \in \langle\mathbb{A}\rangle(Act \times \Pi)$ an **evolution**. The relation $\rightarrow$ is inductively defined by the following:

$$\begin{gathered} (\textbf{Tau})\ \forall P.\ И c.\ \tau.P \overset{c.\tau}{\rightarrow} P \quad (\textbf{Out})\ \forall a, b, P.\ И c.\ \overline{a}b.P \overset{c.\overline{a}b}{\rightarrow} P \\ (\textbf{In})\ \forall a.\ И c.\ \forall P.\ a\langle c\rangle.P \overset{c.ac}{\rightarrow} P \quad (\textbf{Par1})\ \forall P, Q.\ И c.\ \forall \alpha, P'.\ \frac{P \overset{c.\alpha}{\rightarrow} P'}{P \mid Q \overset{c.\alpha}{\rightarrow} P' \mid Q} \\ (\textbf{Com1})\ \forall P, Q, a.\ И c.\ \forall b, P', Q'.\ \frac{P \overset{c.\overline{a}b}{\rightarrow} P' \quad Q \overset{c.ac}{\rightarrow} Q'}{P \mid Q \overset{c.\tau}{\rightarrow} \nu\langle b\rangle.(P' \mid Q'\{b/c\})} \\ (\textbf{Open})\ \forall a.\ И b.\ \forall P, P'.\ И c.\ \frac{P \overset{c.\overline{a}b}{\rightarrow} P'}{\nu\langle b\rangle P \overset{b.\overline{a}b}{\rightarrow} P'} \\ (\textbf{Res})\ \forall \alpha, c.\ И d.\ \forall P, P'.\ \frac{P \overset{c.\alpha}{\rightarrow} P'}{\nu\langle d\rangle P \overset{c.\alpha}{\rightarrow} \nu\langle d\rangle P'} \quad (\textbf{Rep})\ \forall \alpha, c, P, Q.\ \frac{P \mid !P \overset{c.\alpha}{\rightarrow} Q}{!P \overset{c.\alpha}{\rightarrow} Q} \end{gathered} \tag{6}$$

We omit symmetric versions (**Par2**), (**Close2**), and (**Com2**). As before quantifiers order encodes freshness information. For example in (**Res**), $И d$ appears after $\forall \alpha, c$ and this ensures that d is fresh for α and c. The use of colour will be made clear later.

Note the rule (**Com1**). When $b{=}c$ it is interaction involving bound output and yields $\nu\langle b\rangle.(P' \mid Q')$. Otherwise, it involves free output and yields $P' \mid Q'\{b/c\}$ (up to vacuous bindings, for example as generally included in structural congruence).

$$\begin{aligned} P \simeq Q \implies & И c.\ \forall \alpha, P'.\ P \overset{c.\alpha}{\rightarrow} P' \implies \\ & \begin{cases} \exists Q'.\ Q \overset{c.\alpha}{\rightarrow} Q' \wedge P' \simeq Q' & \text{if } \alpha = \overline{a}b \text{ or } \alpha = \tau \\ \exists Q'.\ \forall b.\ Q \overset{c.\alpha}{\rightarrow} Q' \wedge P'\{b/c\} \simeq Q'\{b/c\} & \text{if } \alpha = ac \end{cases} \end{aligned} \tag{7}$$

```
(* pi-calculus a la FM
   We use FreshOCaml abstractions for dynamic
   allocation
   *)

type atom = unit name;;

type proc = Nil
| Snd of atom*atom*proc | Rcv of atom*(<<atom>>proc)
| Par of proc*proc | New of <<atom>>proc
| Rep of proc;;

let rec subs a b p = match p with
  Snd(x,y,p)    ->
     Snd(subsA a b x,subsA a b y,subs a b p)
| Rcv(x,<<n>>p) ->
     Rcv(subsA a b x,<<n>>(subs a b p))
| Par(p1,p2)    -> Par(subs a b p1,subs a b p2)
| New(<<n>>p)   -> New(<<n>>(subs a b p))
| Rep(p)        -> Rep(subs a b p)
| Nil           -> Nil
and  subsA a b x = if x=a then b else x;;

type act = Actt | Acto of atom*atom
         | Acti of atom*atom;;
type evol = <<atom>>(act*proc);;

let par_rule q = function
  <<c>>(Actt,p')      -> <<c>>(Actt,Par(p',q))
| <<c>>(Acto(a,b),p') -> <<c>>(Acto(a,b),Par(p',q))
| <<c>>(Acti(a,b),p') -> <<c>>(Acti(a,b),Par(p',q));;

let com_rule  = function
 <<c>>((Acto(a1,b1),p1'),(Acti(a2,b2),p2')) ->
  if a1=a2 then
[(<<c>>(Actt,New(<<c>>(Par(p1',subs b2 b1 p2'))))))]
          else []
| <<c>>((Acti(a1,b1),p1'),(Acto(a2,b2),p2')) ->
   if a1=a2 then
[(<<c>>(Actt,New(<<c>>(Par(subs b1 b2 p1',p2'))))))]
           else []
| _ -> [];;

let res_open_rule b = function
  <<c>>(Acto(a,b'),p') ->
   if b'!=b && a!=b then [(<<c>>(Acto(a,b'),p'))]
                    else if b'=b && a!=b then
 [(<<b>>(Acto(a,b),p'))] else []
| <<c>>(Acti(a,b'),p') ->
    if a!=b then [(<<c>>(Acti(a,b'),p'))] else []
| <<c>>(Actt,p')       -> [(<<c>>(Actt,p'))];;

let rec multiply_lists l1 l2 =
  match (l1, l2) with
  ([], l2)         -> []
| ([a1],a2::l2)    -> (a1,a2)::(multiply_lists [a1] l2)
| ([a1],[])        -> []
| (a1::a2::l1, l2) -> (multiply_lists [a1] l2) @
                      (multiply_lists (a2::l1) l2);;

let flatten_map f l = List.flatten (List.map f l);;

let unify_names = function (<<a>>x,<<b>>y) ->
 let c=fresh in
  <<c>>((swap c and a in x),(swap c and b in y));;

let rec evol_of = function
  Nil           -> [ ]
| Snd(a,b,p)    ->
   let c=fresh in [(<<c>>(Acto(a,b),p))]
| Rcv(a,<<b>>p) -> [(<<b>>(Acti(a,b),p))]
| New(<<b>>p)   ->
   flatten_map (res_open_rule b) (evol_of p)
| Par(p,q)      ->
   let (evolp,evolq) = (evol_of p,evol_of q)
    in (List.map (par_rule q) evolp)
     @ (List.map (par_rule p) evolq)
     @ (flatten_map com_rule (List.map unify_names
        (multiply_lists evolp evolq)))
| Rep(p)        ->
   let evolp = evol_of p
    in (List.map (par_rule (Rep(p))) evolp)
     @ (List.map (par_rule (Rep(p)))
         (flatten_map com_rule (List.map unify_names
                        (multiply_lists evolp evolp)))
);;

let rec list_remove a = function [] -> []
| x::l' -> if a=x then (list_remove a l')
                  else a::(list_remove a l');;

let rec names_in = function
  Nil           -> []
| Snd(a,b,p)    -> a::b::(names_in p)
| Rcv(a,<<n>>p) -> a::(list_remove n (names_in p))
| Rep(p)        -> names_in p;
| New(<<n>>p)   -> list_remove n (names_in p)
| Par(p,q)      -> (names_in p) @ (names_in q);;

let rec is_bisim p q =
  (is_bisims4(evol_of p, evol_of q)) &&
  (is_bisims4(evol_of q, evol_of p))
and
is_bisims4 (l1,l2) = List.for_all (function e1 ->
   List.exists (function e2 ->
     is_bisims2(unify_names(e1,e2)) l2)) l1
and
is_bisims2  = function
  ((Actt,p),(Actt,q)) -> is_bisim p q
| ((Acto(a1,b1),p),(Acto(a2,b2),q)) ->
      if (a1=a2 && b1=b2) then (is_bisim p q)
                         else false
| ((Acti(a1,b1),p),(Acti(a2,b2),q)) ->
      if a1=a2 && b1=b2 then
      ((is_bisim p q)  &&
       (List.for_all (function x -> x)
        (List.map (function b ->
         is_bisim (subs b1 b p) (subs b2 b q))
         ((names_in p) @ (names_in q))))
      )                 else false
| _ -> false;;

(* A brief demonstration... *)
let (a,b,u) = (fresh, fresh, fresh);;
let p = Rcv(a,u,Nil);;
let q = Snd(a,b,Nil);;
evol_of p;;
evol_of q;;
evol_of (Par(p,q));;
evol_of (New(<<a>>(Par(p,q))));;
is_bisim (Par(p,q)) (Par(q,p));;
```

Fig. 3. pi-calculus using FreshOCaml abstractions for name-generation.

The definitions (6) and (7), and (2) and (4), may look different—but Figure 2 and Figure 3 are rather similar; as discussed in the Introdution. In this version, bound output is detected by output of an abstracted channel. We generally call these abstractions `c` or `b` and they appear at the head of an evolution looking like this: `<<c>>evol`. The relevant cases are:

- `com_rule`, which corresponds to (**Com1**) and its trick of putting possibly vacuous restrictions on terms (in the case that $b = c$ the restriction is not vacuous and we obtain the effect of (**Close**) in (2))
- The clause for `New` in `evol_of` generates a fresh name `b` which is passed to `res_open_rule` which then carries out an equality check `b'=b` to identify bound output.

These points are highlighted in the text.

4 Pi calculus by Sewell

Sewell's pi-calculus is based on a concrete set of contexts with respect to which to choose fresh names. The datatype of processes is identical to (1), but actions are like in (5):

$$\Pi \stackrel{\text{def}}{=} 1 + \mathbb{A}^2 \times \Pi + \mathbb{A}^2 \times \Pi + \Pi^2 + \mathbb{A} \times \Pi \quad Act \cong 1 + \mathbb{A}^2 + \mathbb{A}^2. \tag{8}$$

Name-substitution is capture-avoiding.

The labelled transition relation is $\rightarrow \subseteq \text{fin}(\mathbb{A}) \times \Pi \times Act \times \Pi$. We write

$$A \vdash P \xrightarrow{\alpha} P' \quad \text{for} \quad \langle A, P, \alpha, P' \rangle \in \rightarrow$$

We identify $A \vdash P \xrightarrow{\overline{a}b} P'$ as a bound output by checking $b \notin A$. In the rules below, this effect is obtained using two global conventions to omit side-conditions: first, writing $A \vdash P \xrightarrow{\alpha} P'$ in the conclusion of a rule implies a side-condition which we do not write out that the names in P appear in A. Second, commas above the line indicate disjoint set union, so A, b in (**Open**) implies a side-condition which we do not write out $b \notin A$. These two families of conditions, which we attach to rules as appropriate, are just right to obtain the effect of bound outputs.

The corresponding places in the implementation in Figure 4 are of the form `if List.mem` and are highlighted in blue. The rules are:

$$\begin{gathered}
(\textbf{Out})\ A \vdash \overline{a}b.P \xrightarrow{\overline{a}b} P \\
(\textbf{In})\ A \vdash a\langle c\rangle.P \xrightarrow{ac} P \quad (c \notin A) \qquad (\textbf{Par1})\ \frac{A \vdash P \xrightarrow{\alpha} P'}{A \vdash P \mid Q \xrightarrow{\alpha} P' \mid Q} \\
(\textbf{Com1})\ \frac{A \vdash P \xrightarrow{\overline{a}b} P' \quad A \vdash Q \xrightarrow{ac} Q'}{A \vdash P \mid Q \xrightarrow{\tau} \nu\langle b\rangle.(P' \mid Q'\{b/c\})} \qquad (\textbf{Open})\ \frac{A, b \vdash P \xrightarrow{\overline{a}b} P'}{A \vdash \nu\langle b\rangle P \xrightarrow{\overline{a}b} P'} \quad (a \neq b) \\
(\textbf{Res})\ \frac{A, b \vdash P \xrightarrow{\alpha} P'}{A \vdash \nu\langle b\rangle P \xrightarrow{\alpha} \nu\langle b\rangle P'} \qquad (\textbf{Rep})\ \frac{A \vdash P \mid !P \xrightarrow{\alpha} Q}{A \vdash !P \xrightarrow{\alpha} Q}
\end{gathered} \tag{9}$$

$$A \vdash P \simeq Q \implies \forall \alpha, P'.\ A \vdash P \xrightarrow{\alpha} P' \implies \begin{cases} \exists Q'.\ A \vdash Q \xrightarrow{\alpha} Q' \wedge A \vdash P' \simeq Q' & \text{if } \alpha = \overline{a}b \text{ or } \alpha = \tau \\ \exists Q'.\ \forall b.\ A \vdash Q \xrightarrow{\alpha} Q' \wedge A \vdash P'\{b/c\} \simeq Q'\{b/c\} & \text{if } \alpha = ac \end{cases} \tag{10}$$

```
(* pi-calculus a la Sewell *)

type atom = unit name;;

type proc = Nil
| Snd of atom*atom*proc | Rcv of atom*(<<atom>>proc)
| Par of proc*proc | New of <<atom>>proc
| Rep of proc;;

let rec subs a b p = match p with
  Snd(x,y,p)    ->
    Snd(subsA a b x,subsA a b y,subs a b p)
| Rcv(x,<<n>>p) -> Rcv(subsA a b x,<<n>>(subs a b p))
| Par(p1,p2)    -> Par(subs a b p1,subs a b p2)
| New(<<n>>p)   -> New(<<n>>(subs a b p))
| Rep(p)        -> Rep(subs a b p)
| Nil           -> Nil
and  subsA a b x = if x=a then b else x;;

let rec list_remove a = function
  [] -> []
| x::l' -> if a=x then (list_remove a l')
                  else a::(list_remove a l');;

let rec names_in = function
  Nil           -> []
| Snd(a,b,p)    -> a::b::(names_in p)
| Rcv(a,<<n>>p) -> a::(list_remove n (names_in p))
| Rep(p)        -> names_in p;
| New(<<n>>p)   -> list_remove n (names_in p)
| Par(p,q)      -> (names_in p) @ (names_in q);;

type evol = Etau of proc | Esnd of atom*atom*proc
          | Ercv of atom*(<<atom>>proc);;

let par_rule q = function
  Etau (p')         -> Etau(Par(p',q))
| Esnd (a,b,p')     -> Esnd(a,b,Par(p',q))
| Ercv (a,<<b>>p')  -> Ercv(a,<<b>>(Par(p',q)));;

let com_rule aset = function
  (Esnd (a1,b1,p1'), Ercv (a2,<<b2>>p2')) ->
    if a1=a2 then
      if (List.mem b1 aset) then
        (* if it's a free output *)
        [Etau(Par(p1',subs b2 b1 p2'))]
                              else
        (* if it's a bound output *)
        [Etau(New(<<b2>>(Par(subs b1 b2 p1',p2'))))]
             else []
| (Ercv (a1,<<b1>>p1'), Esnd (a2,b2,p2')) ->
    if a1=a2 then
      if (List.mem b2 aset) then
        [Etau(Par(subs b1 b2 p1',p2'))]
                              else
        [Etau(New(<<b1>>(Par(p1',subs b2 b1 p2'))))]
             else []
| _ -> [];;

let open_rule b = function
(* Could be part of res_rule *)
   (Esnd (a,b',p')) -> if b=b' && a!=b
                         then [Esnd(a,b,p')] else []
| _ -> [];;

let res_rule b = function
  Esnd (a,b',p')  -> if b!=b' && a!=b
                       then [Esnd(a,b',p')] else []
| Ercv (a,phat')  -> if a!=b then
             [Ercv(a,phat')] else []
| Etau (p')       -> [Etau(p')];;

let rec multiply_lists l1 l2 =
  match (l1, l2) with
  ([], l2)         -> []
| ([a1],a2::l2)    -> (a1,a2)::(multiply_lists [a1] l2)
| ([a1],[])        -> []
| (a1::a2::l1, l2) -> (multiply_lists [a1] l2)
                    @ (multiply_lists (a2::l1) l2);;

let flatten_map f l = List.flatten (List.map f l);;

let rec sublist = function
  ([],l2)    -> true
| (a::tl,l2) -> (List.mem a l2) && (sublist(tl,l2));;

let rec evol_of aset = function
  Nil            -> [ ]
| Snd(a,b,p)     -> if sublist(a::b::(names_in p),aset)
                      then [Esnd(a,b,p)] else []
| Rcv(a,<<b>>p') ->
  if sublist(a::(names_in p'),b::aset)
    then [Ercv(a,<<b>>p')] else []
| New(<<b>>p)    ->
   (flatten_map (open_rule b) (evol_of (b::aset) p))@
   (flatten_map (res_rule b) (evol_of (b::aset) p))
| Par(p,q)       ->
let (evolp,evolq) = (evol_of aset p,evol_of aset q)
  in (List.map (par_rule q) evolp)
   @ (List.map (par_rule p) evolq)
   @ (flatten_map (com_rule aset)
                  (multiply_lists evolp evolq))
| Rep(p)        ->
    let evolp = evol_of aset p
     in (List.map (par_rule (Rep(p))) evolp)
      @ (List.map (par_rule (Rep(p)))
          (flatten_map com_rule aset
    (multiply_lists evolp evolp))
);;

let rec is_bisim aset p q =
  (is_bisims4(aset,evol_of aset p, evol_of aset q)) &&
  (is_bisims4(aset,evol_of aset q, evol_of aset p))
and
is_bisims4 (aset,l1,l2) =
  (List.for_all (function e1 ->
   List.exists  (function e2 ->
    is_bisims2(aset,e1,e2)) l2) l1)
and
is_bisims2  = function
  (aset,Etau(p),Etau(q))              ->
    (is_bisim aset p q)
| (aset,Esnd(a1,b1,p),Esnd(a2,b2,q)) ->
    if a1!=a2 then false
              else
      if (List.mem b1 aset) && (List.mem b2 aset)
   then   (* So two bound outputs *)
      let c=fresh in is_bisim (c::aset)
        (swap c and b1 in p) (swap c and b2 in q)
   else if not (List.mem b1 aset) &&
           not (List.mem b2 aset) && b1=b2
   then   (* Two free outputs *)
     (is_bisim aset p q)
   else   (* Win some, lose some *) false
| (aset,Ercv(a1,<<c1>>p),Ercv(a2,<<c2>>q)) ->
    (if a1=a2 then let c=(fresh:atom) in
    (is_bisim (c::aset)
              (swap c and c1 in p)
              (swap c and c2 in q))
              else false)
  && List.for_all (function x -> x)
       (List.map (function b ->
 is_bisim (b::aset) (subs c1 b p)
                    (subs c2 b q))
          ((names_in p) @ (names_in q))
       )
| _ -> false;;

(* A brief demonstration... *)
let (a,b,u) = (fresh, fresh, fresh);;
let p = Rcv(a,u,Nil);;
let q = Snd(a,b,Nil);;
evol_of [] p;;  (* No evolutions *)
evol_of [a,b,u] p;;  (* That's better *)
evol_of [a,b,u] q;;
evol_of (Par(p,q));;
evol_of (New(<<a>>(Par(p,q))));;
is_bisim (Par(p,q)) (Par(q,p));;
```

Fig. 4. pi-calculus using an explicit environment of known names, as presented by Sewell.

5 Pi calculus in HOAS

Miller and Tiu's implementation of the pi-calculus is based on a new logic with a ∇-quantifier built-in, which creates a fresh name. Renaming is also built-in; function types and β-reduction. For the case of β-reduction for a fresh name generated by ∇, the reduction is what Miller calls a β_0-reduction, which can be thought of as a renaming of a name for a fresh name.

We believe β_0-reduction is susceptible to an FM-style semantics. This is for other, future, work.

In Generic Judgements [5] Miller and Tiu had not yet reconciled ∇ with inductive definitions.[1] Accordingly they hypothesise a type Π with constructors of type $\mathbb{A} \to \mathbb{A} \to \Pi \to \Pi$ for output, $\mathbb{A} \to (\mathbb{A} \to \Pi) \to \Pi$ for input, $\Pi \to \Pi \to \Pi$ for parallel composition, $(\mathbb{A} \to \Pi) \to \Pi$ for restriction, and $\Pi \to \Pi$ for replication. Input and output actions have type $\mathbb{A} \to \mathbb{A} \to \mathit{Act}$ for another type Act.

There are two labelled transition relations $\to : \Pi \to \mathit{Act} \to \Pi \to \mathbb{B}$ and $\rightharpoonup : \Pi \to (\mathbb{A} \to \mathit{Act}) \to (\mathbb{A} \to \Pi) \to \mathbb{B}$. The first is used for transitions not involving bound atoms, the second is used for transitions which do involve bound atoms. The rules for deducing transitions are then:

$$
\begin{array}{c}
(\mathbf{Out})\ \overline{a}b.P \xrightarrow{\overline{a}b} P \qquad (\mathbf{In})\ a\langle f\rangle \xrightarrow{a} f \\[2ex]
(\mathbf{Par1free})\ \dfrac{P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q} \qquad (\mathbf{Par1bound})\ \dfrac{P \xrightarrow{\alpha} f'}{P \mid Q \xrightarrow{\alpha} \lambda n.(f'n \mid Q)} \\[2ex]
(\mathbf{Com1})\ \dfrac{P \xrightarrow{a} f' \quad Q \xrightarrow{\overline{a}} g'}{P \mid Q \xrightarrow{\tau} \nu\lambda n.(f'n \mid g'n)} \qquad (\mathbf{Open})\ \dfrac{\nabla m.\ (fm \xrightarrow{\overline{a}m} f'm)}{\nu f \xrightarrow{\overline{a}} f'} \\[2ex]
(\mathbf{Res})\ \dfrac{\nabla m.\ fm \xrightarrow{\alpha} f'm}{\nu f \xrightarrow{\alpha} \nu f'} \qquad (\mathbf{Rep})\ \dfrac{P \mid !P \xrightarrow{\alpha} Q}{!P \xrightarrow{\alpha} Q}
\end{array}
\tag{11}
$$

We write f' and g' for lambda-abstractions which when applied to atoms-arguments a become processes such as $P' = f'a$ and $Q' = g'a$. The colours in (**Open**) are explained later in a discussion of `open_rule` and `evol_of`; the f and m here correspond precisely to `f` and `m` in `evol_of`.

The transition relation can be modelled using $\forall$ instead of ∇. To express bisimulation ∇ is essential, since it allows us to choose a fresh name for bound output:

$$
\begin{array}{l}
P \simeq Q \implies \forall \alpha, P'.\ P \xrightarrow{\alpha} P' \implies \exists Q'.\ Q \xrightarrow{\alpha} Q' \wedge P' \simeq Q' \\[1ex]
\qquad \forall a, f'.\ P \xrightarrow{\overline{a}} f' \wedge P \simeq Q \implies \exists g'.\ \nabla b.\ Q \xrightarrow{\overline{a}} g' \wedge f'b \simeq g'b \\[1ex]
\qquad \forall a, f'.\ P \xrightarrow{a} f' \wedge P \simeq Q \implies \exists g'.\ \forall b.\ Q \xrightarrow{a} g' \wedge f'b \simeq g'b
\end{array}
\tag{12}
$$

There are two ways of implementing this definition depending as whether we pass function-types in the theory to function types in the programming language, or not. After all, we never do any real calculations with atoms (besides generating them with ∇ and using β_0-reduction to pass them around); therefore the f, g, f', and g' of higher type which we see in the definitions are *always* of the form $\lambda n.P\{a/n\}$ for some P and n. If we model what the compiler actually does with these

[1] This is a technical but apparently non-trivial exercise on which Tiu has made progress.

```
(* pi-calculus a la Miller and Tiu,
   delegating function types directly to FreshOCaml *)

type atom = unit name;;

type proc = Nil
| Snd of atom*atom*proc | Rcv of atom*(atom->proc)
| Par of proc*proc | New of (atom->proc)
| Rep of proc;;

let rec subs a b p = match p with
  Snd(x,y,p)    ->
    Snd(subsA a b x,subsA a b y,subs a b p)
| Rcv(x,f)      -> Rcv(subsA a b x,
    function n -> let m=fresh in
      subs m b ((swap m and a in f) n))
 (* Here we ensure that n is nabla-quantified by
    explicitly renaming a to m to avoid clash *)
| Par(p1,p2)    -> Par(subs a b p1,subs a b p2)
| New(f)        -> New(function n ->
    let m=fresh in
          subs m b ((swap m and a in f) n))
   (* Here we ensure that n is nabla-quantified by
      explicitly renaming a to m to avoid clash *)
| Rep(p)        -> Rep(subs a b p)
| Nil           -> Nil
and subsA a b x = if x=a then b else x;;

type evol =
  Etau of proc | Esnd of atom*atom*proc
| Ebsnd of atom*(atom->proc)
| Ercv of atom*(atom->proc);;

let par_rule q = function
  Etau (p')     -> Etau(Par(p',q))
| Esnd (a,b,p') -> Esnd(a,b,Par(p',q))
| Ebsnd (a,f)   -> Ebsnd(a,function b -> (Par(f b,q)))
| Ercv (a,f)    -> Ercv(a,function b ->(Par(f b,q)));;

let com_rule  = function
  (Esnd (a1,b1,p1'), Ercv (a2,f2')) ->
    if a1=a2 then [Etau(Par(p1',f2' b1))] else []
| (Ercv (a1,f1'), Esnd (a2,b2,p2')) ->
    if a1=a2 then [Etau(Par(f1' b2,p2'))] else []
| (Ebsnd (a1,f1'), Ercv (a2,f2'))   ->
    if a1=a2 then [Etau(New(function n ->
                              (Par(f1' n,f2' n))))]
            else []
| (Ercv (a1,f1'), Ebsnd (a2,f2')) ->
      if a1=a2 then [Etau(New(function n ->
                                (Par(f1' n,f2' n))))]
               else []
| _ -> [];;

let open_rule m = function
  (Esnd (a,b',p')) -> if m=b'
   (* inductively guaranteed by fresh && a!=m *)
 then [Ebsnd(a,function n -> (subs m n p'))]
 else []
| _ -> [];;

let res_rule b = function
  Esnd (a,b',p')  -> if b!=b' && a!=b then
                       [Esnd(a,b',p')] else []
| Ebsnd (a,phat') -> if a!=b then [Ebsnd(a,phat')]
                             else []
| Ercv (a,phat')  -> if a!=b then [Ercv(a,phat')]
                             else []
| Etau (p')       -> [Etau(p')];;

let rec multiply_lists l1 l2 = match (l1, l2) with
  ([], l2)         -> []
| ([a1],a2::l2)    ->
    (a1,a2)::(multiply_lists [a1] l2)
| ([a1],[])        -> []
| (a1::a2::l1, l2) ->
    (multiply_lists [a1] l2)
  @ (multiply_lists (a2::l1) l2);;

let flatten_map f l = List.flatten (List.map f l);;

let rec evol_of = function
  Nil           -> [ ]
| Snd(a,b,p)    -> [Esnd(a,b,p)]
| Rcv(a,f)      -> [Ercv(a,f)]
| New(f)        -> let m=(fresh:atom) in
    (flatten_map (open_rule m) (evol_of (f m)))@
    (flatten_map (res_rule m) (evol_of (f m)))
| Par(p,q)      ->
    let (evolp,evolq) = (evol_of p,evol_of q)
     in (List.map (par_rule q) evolp)
      @ (List.map (par_rule p) evolq)
      @ (flatten_map com_rule
          (multiply_lists evolp evolq))
| Rep(p)        ->
    let evolp = evol_of p
     in (List.map (par_rule (Rep(p))) evolp)
      @ (List.map (par_rule (Rep(p)))
          (flatten_map com_rule
    (multiply_lists evolp evolp))
);;

let rec list_remove a = function
  [] -> []
| x::l' -> if a=x then (list_remove a l')
                  else a::(list_remove a l');;

let rec names_in = function
  Nil           -> []
| Snd(a,b,p)    -> a::b::(names_in p)
| Rcv(a,f)      -> let m=fresh in
    a::(list_remove m (names_in (f m)))
| Rep(p)        -> names_in p;
| New(f)        -> let m=fresh in
    list_remove m (names_in (f m))
| Par(p,q)      -> (names_in p) @ (names_in q);;

let rec is_bisim p q =
  (is_bisims4(evol_of p, evol_of q)) &&
  (is_bisims4(evol_of q, evol_of p))
and
is_bisims4 (l1,l2) =
 (List.for_all (function e1 ->
  List.exists (function e2 ->
    is_bisims2(e1,e2)) l2) l1)
and
is_bisims2  = function
  (Etau(p),Etau(q))             -> (is_bisim p q)
| (Esnd(a1,b1,p),Esnd(a2,b2,q)) ->
     if a1=a2 && b1=b2 then
        (is_bisim p q) else false
| (Ercv(a1,fp),Ercv(a2,fq)) ->
     (if a1=a2 then
let m=(fresh:atom) in (is_bisim (fp m) (fq m))
              else false)
  && List.for_all (function x -> x)
       (List.map
          (function b -> is_bisim (fp b) (fq b))
            ((names_in p) @ (names_in q))
       )
| (Ebsnd(a1,fp),Ebsnd(a2,fq)) ->
     if a1=a2 then
let m=(fresh:atom) in (is_bisim (fp m) (fq m))
              else false
| _ -> false;;

(* A brief demonstration... *)
let (a,b,u) = (fresh, fresh, fresh);;
let p = Rcv(a,function n -> subs a n Nil);;
let q = Snd(a,b,Nil);;
evol_of p;;
evol_of q;;
evol_of (Par(p,q));;
evol_of (New(a,Par(p,q)));;
is_bisim (Par(p,q)) (Par(q,p));;
```

Fig. 5. pi-calculus following Generic Judgements, delegating function types to FreshOCaml function types.

names we obtain precisely Milner's program from §2. For example in the rule for input, f must be of the form $\lambda b.P$, so we obtain precisely Milner's rule for input. In this implementation of the rules, the actual computation taking place is precisely equivalent to Milner's system.

On the other hand we can pass function-types to the programming language, letting it take care of α-renaming and substitution for us. We then obtain the program illustrated in Figure 5.

In this program abstractions are function types `atom->proc`. Most of the time (see below) we achieve substitution with function application to a fresh name (β_0-reduction, we generally call them `m`) or to a possibly not fresh name (β-reduction, on variables, we generally call them `n`). As the reader probably knows it is up to us to program substitution into every process: the term `New(function a -> if a=b then P1 else P2)` is an *exotic term*, not corresponding to a textual substitution. `subs` is useful to build these processes; see the second line of the brief demonstration (in blue, though admittedly substituting in `Nil` is no great challenge; but because we have `subs`, see the trick discussed below, any `p'` is admissible and even a variable of type `proc`).

We still use `fresh` to generate fresh names in `subs` and `is_bisims2`. We also use an interesting FreshOCaml trick in the clauses for `Rcv` and `New` in `subs`, without which it would be hard to implement `subs` at all: we use swaps to rename atoms `m` and `a` *in a function* `f`, to avoid name-clash. Without the FM dynamic duo of name-generation `fresh` and name-swapping `swap`, this would be harder to express.

We must now ask why we define `subs` if we have functions and application as discussed above. We need it in `open_rule` because we interpret (**Open**) very literally in `evol_of` by applying `f` to `m`.

Miller and Tiu use Logic Programming; the logic of Generic Judgements (or logics derived from it) is intended to be implemented directly as a logic programming language, so that the mathematical specification given [5] *is* the program.

Thus our implementation misrepresents Miller and Tiu's work by implementing it in a functional programming language. One could argue that Generic Judgements might well be implemented in some functional language, so that the programs we see are a fair reflection of the actual computation—but we must also remember that proof-search in a logic-programming language can be more 'intelligent' than that. For example, in the presence of matching $[x{=}y]P$ a logical system might immediately add $x{=}y$ into the context where a functional implementation might systematically check all possibilities of x and y.

6 Evaluation and related work

The goal of this paper is to compare mathematical formalisms of the pi calculus, by implementing them. Our implementations mainly use FreshOCaml, an extension of OCaml based on FM theory and the 'fresh' operator $\mathcal{N}$ [10]. We now explain the background for this choice of language.

All implementations require fresh names, and generate them for the same reasons: for capture-avoiding substitution e.g. $(\nu x.P)\{x/y\} = \nu x'.(P\{x'/x\}\{x/y\})$; for clash-avoidance in the LTS rules `par`, `com` and `res`; and for comparing one bound transition with another in bisimulation.

To implement fresh name generation, we first used OCaml references, as in `let n=ref()`. Next we used FreshOCaml abstractions `let n=(fresh:atom)` and implicitly in pattern-matching `<<n>>p`. Both solutions amount to keeping a global state and deducing from it a free name. We could instead have implemented this global state 'manually', perhaps with a lazy free list or a counter, passed it explicitly to each function.

Another possibility is to generate fresh names *locally* with respect to the processes in question, rather than globally. In effect, we implement $(x \notin n(P))$ rather than *(x fresh)*.

Sewell's version is close to the suggestions above: the explicit state of known names makes it easy to choose a fresh one.

We are not proud of any of our solutions to the above issues. FreshOCaml seemed the least bad solution. Although it involves a global state, this global state at least has an elegant local mathematical treatment through the fresh operator И [10]. The FreshOCaml implementation of substitution (the function `subs` in Figure 2) may be unfamiliar to most programmers, but it is more concise than the OCaml version (Figure 1).

This paper only provides half a model-checker – the half to do with states and transitions. It does not deal with an entire transition graph, nor with collapsing equivalent states. This makes it unsuitable for terms with replication, and wildly inefficient for weak bisimulation (which we therefore did not implement). For collapsing techniques specific to the pi calculus, see [8]. On the topic of collapsing states, we also mention History-Dependent Automata and HAL [11]. HD-automata use de-Bruijn notation; they represent the emission of a bound name $\nu x.\overline{u}\,x.P \stackrel{\overline{u}(x)}{\rightarrow} P$ by inserting a new index number 1 and shifting all the others along. We had hoped to implement HD-automata in the current paper, perhaps even in FM. However this turned out to miss the point. The chief contribution of HD-automata is its collapse of equivalent states.

The established model-checker for the pi calculus is the Mobility Workbench [12]. One of its features is its use of *symbolic bisimulation* [4, 7] rather than the late bisimulation used in this paper. Symbolic bisimulation avoids substitution over all free names, and instead considers only substitutions that may affect behaviour. We see our current work as complementary to the Mobility Workbench. So would pi model-checkers be better if they use a different formalism for the pi calculus? No, all the formalisms yielded more or less the same code.

References

1. FreshML, *FreshML homepage*, , `http://www.freshml.org`.
2. M. J. Gabbay, *The π-calculus in FM*, Thirty-five years of Automath (Fairouz Kamareddine, ed.), Kluwer, 2003.
3. M. J. Gabbay and A. M. Pitts, *A new approach to abstract syntax with variable binding*, Formal Aspects of Computing **13** (2001), 341–363.
4. Mathew Hennessy and H. Lin, *Symbolic bisimulations*, Theoretical Computer Science **138** (1995), no. 2, 353–389.
5. Dale Miller and Alwen Tiu, *A proof theory for generic judgments: An extended abstract*, Proceedings of LICS 2003, IEEE, June 2003, pp. 118–127.
6. Robin Milner, Joachim Parrow, and David Walker, *A calculus of mobile processes, II*, Information and Computation **100** (1992), no. 1, 41–77.
7. Davide Sangiorgi, *A theory of bisimulation for the pi calculus*, Acta Informatica **33** (1996), 69–96.
8. ———, *On the bisimulation proof method*, Mathematical Structures in Computer Science **8** (1998), no. 5, 447–479.
9. Peter Sewell, *Applied pi - a brief tutorial*, Tech. Report 498, Computer Laboratory, University of Cambridge, 2000.
10. M. R. Shinwell, A. M. Pitts, and M. J. Gabbay, *FreshML: Programming with binders made simple*, Eighth ACM SIGPLAN International Conference on Functional Programming (ICFP 2003), Uppsala, Sweden, ACM Press, August 2003, pp. 263–274.
11. Marco Pistore Ugo Montanari, *Structured coalgebras and minimal hd-automata for the pi-calculus*, Tech. Report 0006-02, IRST-ITC, 2000.
12. Björn Victor and Faron Moller, *The mobility workbench - a tool for the pi calculus*, CAV'94 (David Dill, ed.), Lecture Notes in Computer Science, vol. 818, Springer-Verlag, 1994, pp. 428–440.